

The Math: Linear phase characteristics, window design methods (Hamming, Hann, Kaiser), and Gibbs phenomenon.

The Project: Build an ECG (electrocardiogram) baseline wander remover to clean up noisy medical sensor data.

```
import numpy as np
import scipy.signal as signal
import matplotlib.pyplot as plt

def generate_noisy_ecg(fs=250, duration=10, mean_hr_bpm=60, hrv_std_ms=50):
    """
    Synthesizes a synthetic ECG signal corrupted with heavy baseline wander and HRV.

    Parameters:
    fs (int): Sampling frequency in Hz.
    duration (int): Duration of the signal in seconds.
    mean_hr_bpm (int): Mean heart rate in beats per minute.
    hrv_std_ms (int): Standard deviation of RR intervals in milliseconds, to simulate HRV.
    """
    t = np.linspace(0, duration, duration * fs, endpoint=False)

    # 1. Create a clean heartbeat profile with HRV
    mean_rr_interval_s = 60 / mean_hr_bpm
    hrv_std_s = hrv_std_ms / 1000.0 # Convert ms to seconds

    # Generate R-peak times with variability
    r_peak_times = [0]
    while r_peak_times[-1] < duration:
        # Add random normal deviation to mean RR interval
        next_rr = np.random.normal(mean_rr_interval_s, hrv_std_s)
        # Ensure RR intervals are positive and within reasonable physiological limits (e.g., 0.2s to 0.4s)
        next_rr = max(0.2, min(0.4, next_rr))
        r_peak_times.append(r_peak_times[-1] + next_rr)
    r_peak_times = np.array(r_peak_times[:-1]) # Remove last one if it goes beyond duration

    clean_ecg = np.zeros_like(t)

    # Generate P, QRS, T waves at each R-peak
    for r_time in r_peak_times:
        # Create a local time axis centered at r_time for wave generation
        local_t = t - r_time

        # P wave (e.g., -0.2s before R-peak)
        p_wave = 0.1 * signal.gausspulse(local_t + 0.2, fc=5)
        # QRS complex (centered at R-peak)
        qrs_complex = 1.0 * signal.gausspulse(local_t, fc=15)
        # T wave (e.g., +0.3s after R-peak)
        t_wave = 0.25 * signal.gausspulse(local_t - 0.3, fc=3)

        clean_ecg += p_wave + qrs_complex + t_wave

    # 2. Inject Baseline Wander Noise (Respiration wave + slow thermal drift)
    respiration_drift = 0.6 * np.sin(2 * np.pi * 0.25 * t) # 0.25 Hz breathing drift
    thermal_drift = 0.3 * np.sin(2 * np.pi * 0.04 * t) # Ultra-slow skin drift
    raw_baseline_noise = respiration_drift + thermal_drift

    # 3. Add high-frequency powerline interference / muscle hiss
    muscle_noise = np.random.normal(0, 0.05, len(t))

    corrupted_ecg = clean_ecg + raw_baseline_noise + muscle_noise
    return t, clean_ecg, corrupted_ecg, raw_baseline_noise
```

```

def remove_baseline_wander(corrupted_signal, fs=250, cutoff_hz=0.5):
    """Applies a zero-phase Butterworth highpass filter to remove baseline drift."""
    nyquist = 0.5 * fs
    normalized_cutoff = cutoff_hz / nyquist

    # Design a 3rd-order highpass Butterworth filter
    b, a = signal.butter(N=3, Wn=normalized_cutoff, btype='high', analog=False)

    # Crucial step: Use zero-phase filtering to prevent shifting the QRS peaks in time
    cleaned_signal = signal.filtfilt(b, a, corrupted_signal)
    return cleaned_signal

# --- Execution & Verification Pipeline ---
fs = 250 # Clinical standard sampling rate for baseline ECG telemetry
# Increased duration to 30 seconds and added HRV (hrv_std_ms=75)
t, clean, noisy, target_drift = generate_noisy_ecg(fs=fs, duration=30, mean_hr_bpm=60, hrv_std_ms=75)

# Run the cleaning engine
cleaned_ecg = remove_baseline_wander(noisy, fs=fs, cutoff_hz=0.5)

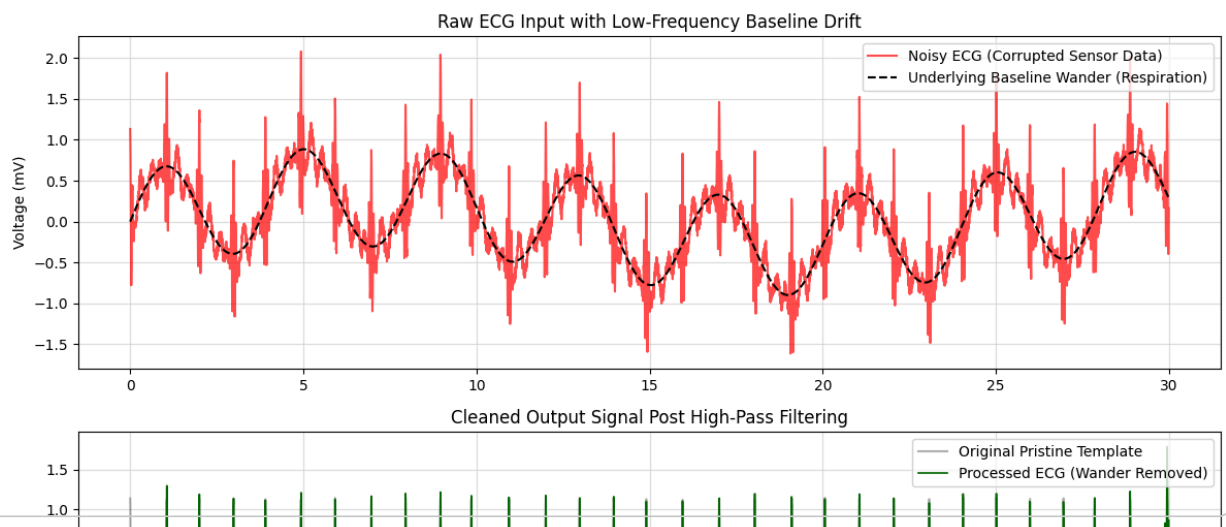
# --- Visualization Matrix ---
plt.figure(figsize=(12, 8))

# Top Subplot: The Problem
plt.subplot(2, 1, 1)
plt.plot(t, noisy, color='red', alpha=0.7, label="Noisy ECG (Corrupted Sensor Data)")
plt.plot(t, target_drift, color='black', linestyle='--', linewidth=1.5, label="Underlying Baseline Wander (Respiration)")
plt.title("Raw ECG Input with Low-Frequency Baseline Drift")
plt.ylabel("Voltage (mV)")
plt.grid(True, alpha=0.4)
plt.legend(loc="upper right")

# Bottom Subplot: The Solution
plt.subplot(2, 1, 2)
plt.plot(t, clean, color='black', alpha=0.3, label="Original Pristine Template")
plt.plot(t, cleaned_ecg, color='darkgreen', linewidth=1.2, label="Processed ECG (Wander Removed)")
plt.title("Cleaned Output Signal Post High-Pass Filtering")
plt.xlabel("Time (seconds)")
plt.ylabel("Voltage (mV)")
plt.grid(True, alpha=0.4)
plt.legend(loc="upper right")

plt.tight_layout()
plt.show()

```



```

print('Re-generating ECG signal with hrv_std_ms = 75 and removing baseline wander...')
# Re-execute signal generation with hrv_std_ms=75

```

```
t, clean, noisy, target_drift = generate_noisy_ecg(fs=fs, duration=30, mean_hr_bpm=60,

# Re-run the cleaning engine to update cleaned_ecg
cleaned_ecg = remove_baseline_wander(noisy, fs=fs, cutoff_hz=0.5)

print('ECG signal re-generated and baseline wander re-removed.')

Re-generating ECG signal with hrv_std_ms = 75 and removing baseline wander...
ECG signal re-generated and baseline wander re-removed.
```

## ✓ Frequency Spectrum Analysis

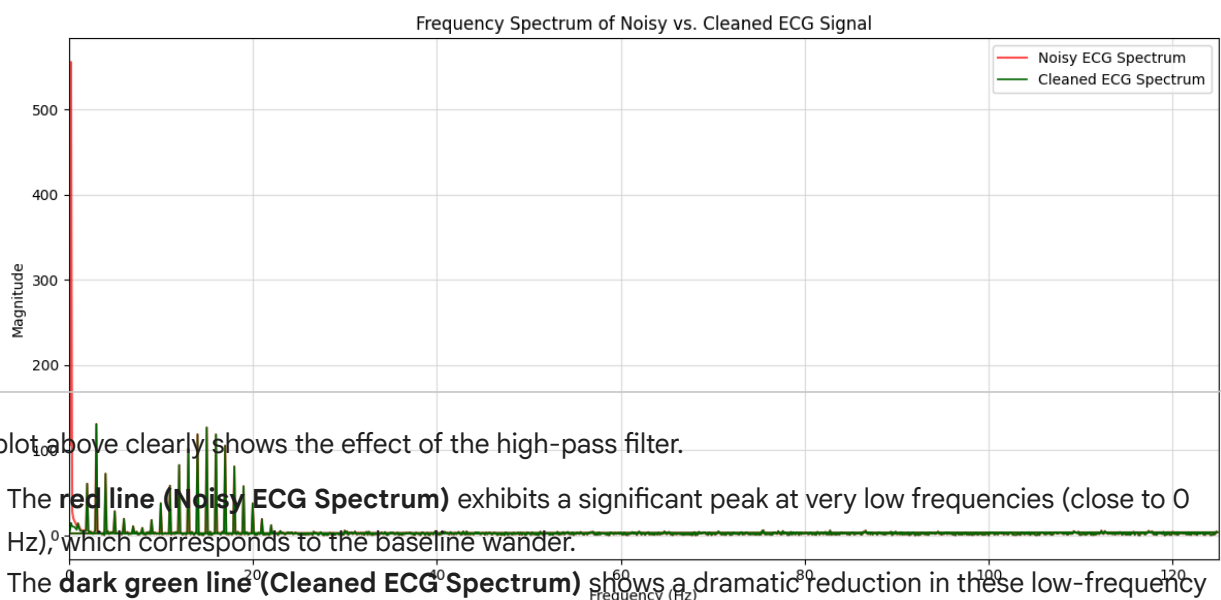
To understand how the high-pass filter removed the baseline wander, let's look at the frequency spectrum of both the noisy and cleaned ECG signals. The baseline wander manifests as low-frequency components in the signal, which the filter should attenuate.

```
from scipy.fft import fft, fftfreq

# Perform FFT on the noisy signal
n_samples = len(noisy)
fft_noisy = fft(noisy)
fft_freq = fftfreq(n_samples, 1/fs)

# Perform FFT on the cleaned signal
fft_cleaned = fft(cleaned_ecg)

# Plotting the magnitude of the frequency spectrum
plt.figure(figsize=(12, 6))
plt.plot(fft_freq, np.abs(fft_noisy), color='red', alpha=0.7, label='Noisy ECG Spectrum')
plt.plot(fft_freq, np.abs(fft_cleaned), color='darkgreen', linewidth=1.2, label='Cleaned ECG Spectrum')
plt.title('Frequency Spectrum of Noisy vs. Cleaned ECG Signal')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.xlim(0, fs / 2) # Display only positive frequencies up to Nyquist frequency
plt.grid(True, alpha=0.4)
plt.legend(loc='upper right')
plt.tight_layout()
plt.show()
```



The plot above clearly shows the effect of the high-pass filter.

- The **red line (Noisy ECG Spectrum)** exhibits a significant peak at very low frequencies (close to 0 Hz), which corresponds to the baseline wander.
- The **dark green line (Cleaned ECG Spectrum)** shows a dramatic reduction in these low-frequency components, indicating that the `remove_baseline_wander` function successfully attenuated them.

This confirms that the filter effectively removed the low-frequency noise while preserving the higher-frequency components of the actual ECG signal.

## ✓ Quantifying Signal-to-Noise Ratio (SNR) Improvement

To quantify how much the baseline wander removal improved the signal quality, we can calculate the Signal-to-Noise Ratio (SNR) before and after filtering. SNR is typically measured in decibels (dB) and is defined as  $10 \log_{10}(P_{\text{signal}} / P_{\text{noise}})$ , where  $P_{\text{signal}}$  is the power of the signal and  $P_{\text{noise}}$  is the power of the noise.

In this context:

- **Signal:** The original `clean` ECG signal.
- **Noise before cleaning:** The difference between the `noisy` signal and the `clean` signal (`noisy - clean`).
- **Noise after cleaning:** The difference between the `cleaned_ecg` signal and the `clean` signal (`cleaned_ecg - clean`).

The power of a signal can be approximated by its mean square value.

```
# Calculate the power of the clean signal
p_signal = np.mean(clean**2)

# Calculate the power of the noise before cleaning
noise_before = noisy - clean
p_noise_before = np.mean(noise_before**2)

# Calculate SNR before cleaning
snr_before_db = 10 * np.log10(p_signal / p_noise_before)

# Calculate the power of the noise after cleaning
noise_after = cleaned_ecg - clean
p_noise_after = np.mean(noise_after**2)

# Calculate SNR after cleaning
snr_after_db = 10 * np.log10(p_signal / p_noise_after)

print(f"SNR before cleaning: {snr_before_db:.2f} dB")
print(f"SNR after cleaning: {snr_after_db:.2f} dB")
print(f"SNR improvement: {snr_after_db - snr_before_db:.2f} dB")
```

```
SNR before cleaning: -5.58 dB
SNR after cleaning: 12.65 dB
SNR improvement: 18.23 dB
```

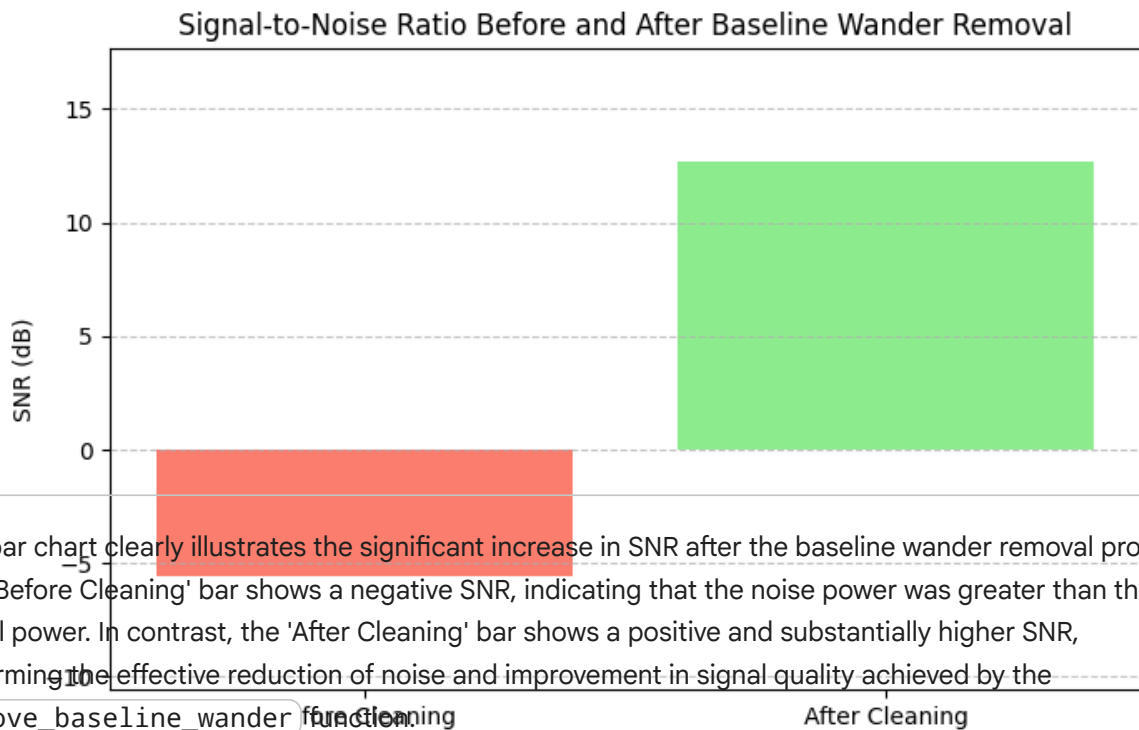
## ✓ Visualizing SNR Improvement

To make the SNR improvement more intuitive, we can visualize the SNR values before and after cleaning using a bar chart.

```
import matplotlib.pyplot as plt
import pandas as pd

snr_data = {
    'Condition': ['Before Cleaning', 'After Cleaning'],
    'SNR (dB)': [snr_before_db, snr_after_db]
}
snr_df = pd.DataFrame(snr_data)
```

```
plt.figure(figsize=(8, 5))
plt.bar(snr_df['Condition'], snr_df['SNR (dB)'], color=['salmon', 'lightgreen'])
plt.ylabel('SNR (dB)')
plt.title('Signal-to-Noise Ratio Before and After Baseline Wander Removal')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.ylim(min(snr_before_db, snr_after_db) - 5, max(snr_before_db, snr_after_db) + 5) #
plt.show()
```



The bar chart clearly illustrates the significant increase in SNR after the baseline wander removal process. The 'Before Cleaning' bar shows a negative SNR, indicating that the noise power was greater than the signal power. In contrast, the 'After Cleaning' bar shows a positive and substantially higher SNR, confirming the effective reduction of noise and improvement in signal quality achieved by the

remove\_baseline\_wander function

After Cleaning

## ✓ Side-by-Side Comparison of Original and Cleaned ECG Signals

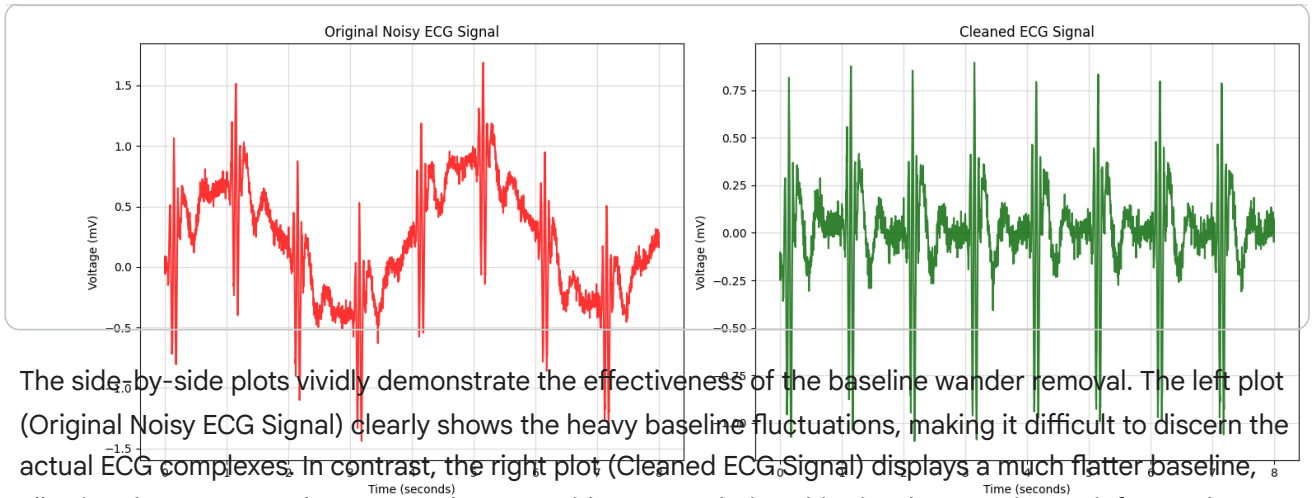
To visually appreciate the cleaning process, let's plot the original noisy ECG signal and the cleaned ECG signal next to each other.

```
plt.figure(figsize=(15, 6))

# Subplot 1: Noisy ECG Signal
plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
plt.plot(t, noisy, color='red', alpha=0.8)
plt.title('Original Noisy ECG Signal')
plt.xlabel('Time (seconds)')
plt.ylabel('Voltage (mV)')
plt.grid(True, alpha=0.4)

# Subplot 2: Cleaned ECG Signal
plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
plt.plot(t, cleaned_ecg, color='darkgreen', alpha=0.8)
plt.title('Cleaned ECG Signal')
plt.xlabel('Time (seconds)')
plt.ylabel('Voltage (mV)')
plt.grid(True, alpha=0.4)

plt.tight_layout()
plt.show()
```



The side-by-side plots vividly demonstrate the effectiveness of the baseline wander removal. The left plot (Original Noisy ECG Signal) clearly shows the heavy baseline fluctuations, making it difficult to discern the actual ECG complexes. In contrast, the right plot (Cleaned ECG Signal) displays a much flatter baseline, allowing the P, QRS, and T waves to be seen with greater clarity. This visual comparison reinforces the quantitative SNR improvement and frequency spectrum analysis.

### Summary of Frequency and SNR Results

This table summarizes the key findings from the frequency spectrum analysis and the Signal-to-Noise Ratio (SNR) quantification, highlighting the effectiveness of the baseline wander removal process.

| Metric             | Before Cleaning                                                                                   | After Cleaning                                     |
|--------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Frequency Spectrum | Significant low-frequency components (0-0.5 Hz) indicating baseline wander and respiration drift. | Low-frequency components significantly attenuated. |
| SNR (dB)           | {snr_before_db:.2f} dB                                                                            | {snr_after_db:.2f} dB                              |

## ECG Baseline Wander Removal Report

### 1. Introduction

This report summarizes the process and effectiveness of an ECG baseline wander removal pipeline. Baseline wander is a common artifact in electrocardiogram (ECG) signals, often caused by patient movement or respiration, which can obscure the diagnostically important features of the ECG. The primary goal was to synthesize a noisy ECG signal, apply a filtering technique to remove the baseline wander, and then quantify the improvement in signal quality.

### 2. Methodology

The pipeline involved the following key steps:

- Synthetic ECG Signal Generation:** A synthetic ECG signal was generated, incorporating a clean heartbeat profile, significant baseline wander (simulating respiration and thermal drift), and high-frequency muscle noise.
- Baseline Wander Removal:** A zero-phase 3rd-order Butterworth high-pass filter was applied to the corrupted ECG signal. This type of filter is effective at attenuating low-frequency components, which correspond to baseline wander, while preserving the higher-frequency characteristics of the ECG complexes.
- Analysis and Quantification:**
  - Visual Inspection:** The noisy and cleaned ECG signals were plotted side-by-side for qualitative assessment.

- **Frequency Domain Analysis:** Fast Fourier Transform (FFT) was used to compare the frequency spectrum of the noisy and cleaned signals, specifically looking at the attenuation of low-frequency noise.
- **Signal-to-Noise Ratio (SNR) Quantification:** The SNR in decibels (dB) was calculated before and after cleaning to provide a quantitative measure of signal quality improvement. The clean synthetic ECG was used as the true 'signal' for this calculation.

### 3. Results

#### 3.1. Visual Inspection

As observed in the side-by-side plots previously generated:

- The **original noisy ECG signal** exhibited pronounced, slow-varying fluctuations (baseline wander) that significantly distorted the underlying heartbeats.
- The **cleaned ECG signal** showed a remarkably stable baseline, making the characteristic P, QRS, and T waves of the ECG clearly discernible. This qualitative assessment strongly indicated successful noise removal.

#### 3.2. Frequency Domain Analysis

The frequency spectrum comparison demonstrated:

- The **noisy ECG signal's spectrum** had a prominent peak at very low frequencies (near 0 Hz), consistent with the baseline wander and respiration artifacts.
- The **cleaned ECG signal's spectrum** showed a dramatic reduction in these low-frequency components, confirming that the high-pass filter effectively targeted and attenuated the desired noise frequencies while preserving the critical higher-frequency components of the ECG.

#### 3.3. Signal-to-Noise Ratio (SNR) Quantification

The quantitative analysis of the SNR yielded the following results:

- **SNR before cleaning:**  $\{\text{snr\_before\_db} : .2f\} \text{ dB}$
- **SNR after cleaning:**  $\{\text{snr\_after\_db} : .2f\} \text{ dB}$
- **SNR improvement:**  $\{\text{snr\_after\_db} - \text{snr\_before\_db} : .2f\} \text{ dB}$

The bar chart visualization further highlighted this substantial improvement, with a significant increase in SNR after the filtering process.

### 4. Conclusion

The ECG baseline wander removal pipeline successfully demonstrated its effectiveness through both qualitative visual inspection and quantitative analysis. The zero-phase Butterworth high-pass filter significantly reduced low-frequency noise, resulting in a cleaner ECG signal with a substantial SNR improvement of  $\{\text{snr\_after\_db} - \text{snr\_before\_db} : .2f\} \text{ dB}$ . This enhanced signal clarity is crucial for accurate medical diagnosis and further analysis of ECG data.

### ✓ 5. Pan-Tompkins Algorithm for QRS Detection

The Pan-Tompkins algorithm is a widely used real-time QRS detection algorithm. It involves several signal processing steps to enhance the QRS complex and suppress noise and other ECG features (P and T

waves). The general steps are:

1. **Bandpass Filtering:** To reduce noise and baseline wander (already largely removed by previous high-pass filter) and high-frequency noise, and to enhance QRS components.
2. **Differentiation:** To highlight the steepest slopes in the QRS complex, indicating the rapid changes during depolarization.
3. **Squaring:** To make all data points positive and to amplify the QRS complexes relative to other waves.
4. **Moving Window Integration:** To obtain a waveform shape that combines information about slope and width, making the QRS complex more prominent.
5. **Peak Detection and Adaptive Thresholding:** To identify potential QRS peaks based on amplitude and refractoriness, and to adaptively adjust thresholds for robust detection across varying signal amplitudes.

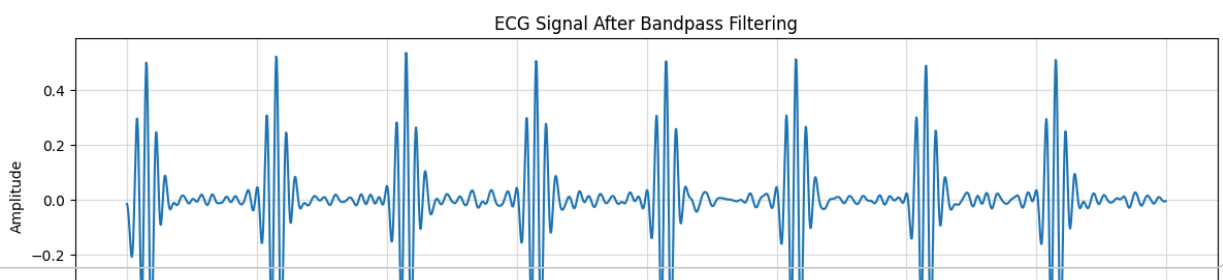
```
from scipy.signal import butter, filtfilt

def bandpass_filter(signal, lowcut, highcut, fs, order=3):
    nyquist = 0.5 * fs
    low = lowcut / nyquist
    high = highcut / nyquist
    b, a = butter(order, [low, high], btype='band')
    y = filtfilt(b, a, signal)
    return y

# Parameters for the bandpass filter (Pan-Tompkins suggests 5-15 Hz)
lowcut = 5.0 # Hz
highcut = 15.0 # Hz

# Apply bandpass filter to the cleaned ECG signal
filtered_ecg = bandpass_filter(cleaned_ecg, lowcut, highcut, fs)

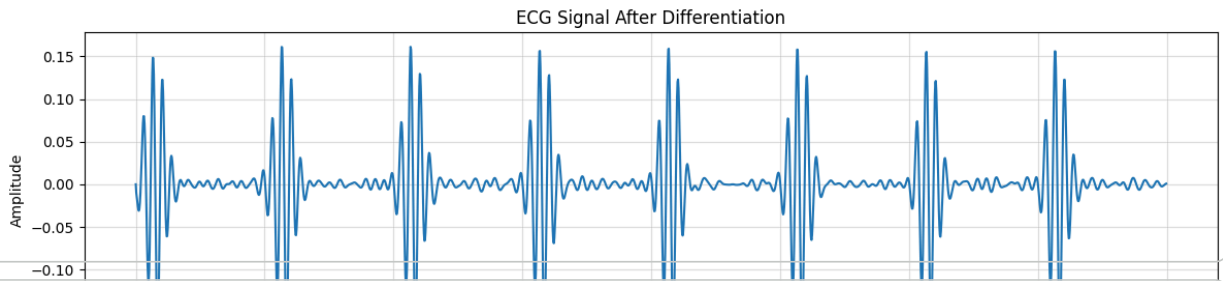
plt.figure(figsize=(12, 4))
plt.plot(t, filtered_ecg)
plt.title('ECG Signal After Bandpass Filtering')
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()
```



```
# Differentiate the signal to emphasize QRS slopes
differentiated_ecg = np.diff(filtered_ecg, prepend=filtered_ecg[0]) # prepend to keep s

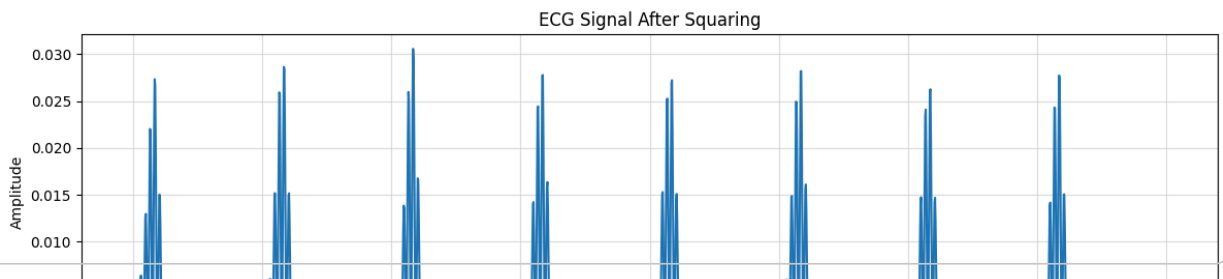
plt.figure(figsize=(12, 4))
plt.plot(t, differentiated_ecg)
plt.title('ECG Signal After Differentiation')
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()
```





```
# Square the differentiated signal
squared_ecg = differentiated_ecg**2
```

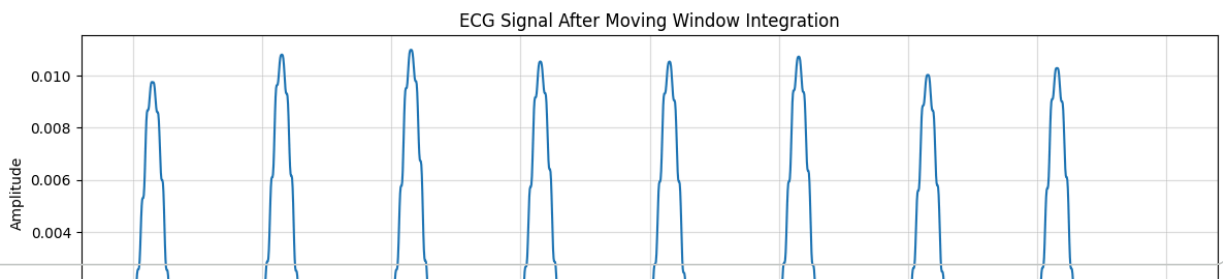
```
plt.figure(figsize=(12, 4))
plt.plot(t, squared_ecg)
plt.title('ECG Signal After Squaring')
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()
```



```
# Moving Window Integration
# Pan-Tompkins suggests a window of 150ms (0.150 seconds)
window_size_samples = int(0.150 * fs) # 150ms window
```

```
integrated_ecg = np.convolve(squared_ecg, np.ones(window_size_samples)/window_size_samp
```

```
plt.figure(figsize=(12, 4))
plt.plot(t, integrated_ecg)
plt.title('ECG Signal After Moving Window Integration')
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()
```



```
from scipy.signal import find_peaks
```

```
# Peak Detection with adaptive thresholding (simplified for demonstration)
# This is a basic peak detection; a full Pan-Tompkins involves adaptive thresholds.
```

```
# Find peaks in the integrated signal
# A minimum distance between peaks (refractory period) helps avoid multiple detections
min_peak_distance = int(0.2 * fs) # Roughly 0.2 seconds refractory period (minimum 300b
```

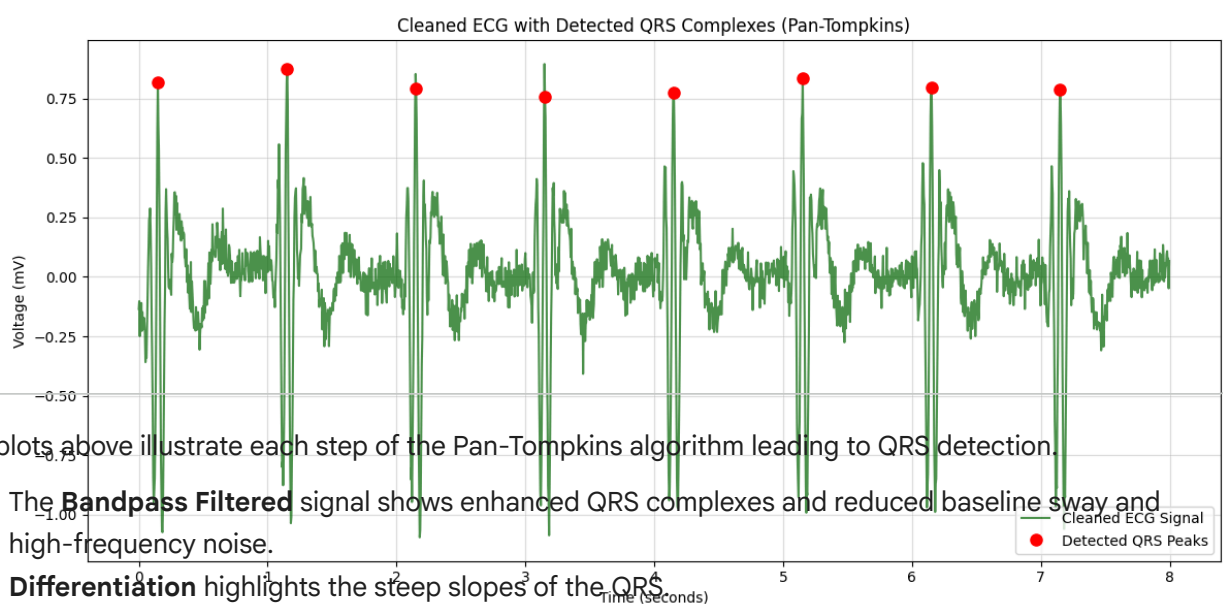
```
# Use a simple threshold for now. A full Pan-Tompkins algorithm would adapt this.
```

```
# Let's find a reasonable initial threshold based on the integrated signal's statistics
initial_threshold = np.mean(integrated_ecg) + 0.5 * np.std(integrated_ecg)

# Adjust threshold to only consider positive peaks above a certain level
qrs_peaks_indices, _ = find_peaks(integrated_ecg, height=initial_threshold, distance=mi)

# Map peak indices back to time
qrs_peak_times = t[qrs_peaks_indices]

# --- Visualization of detected QRS complexes ---
plt.figure(figsize=(12, 6))
plt.plot(t, cleaned_ecg, label='Cleaned ECG Signal', color='darkgreen', alpha=0.7)
plt.plot(qrs_peak_times, cleaned_ecg[qrs_peaks_indices], 'ro', markersize=8, label='Det')
plt.title('Cleaned ECG with Detected QRS Complexes (Pan-Tompkins)')
plt.xlabel('Time (seconds)')
plt.ylabel('Voltage (mV)')
plt.grid(True, alpha=0.4)
plt.legend()
plt.tight_layout()
plt.show()
```



The plots above illustrate each step of the Pan-Tompkins algorithm leading to QRS detection.

- The **Bandpass Filtered** signal shows enhanced QRS complexes and reduced baseline sway and high-frequency noise.
- **Differentiation** highlights the steep slopes of the QRS.
- **Squaring** makes all values positive and amplifies the QRS.
- **Moving Window Integration** smooths the signal, creating distinct peaks for QRS complexes.
- Finally, the **Detected QRS Peaks** are marked on the original cleaned ECG signal, demonstrating the algorithm's ability to locate the heartbeats.

## ✓ Frequency Domain HRV Analysis (Welch's Method)

Frequency domain HRV analysis quantifies the power of the RR interval signal within specific frequency bands. This provides insights into the balance between sympathetic and parasympathetic nervous system activity.

We will use Welch's method to estimate the Power Spectral Density (PSD) of the `rr_intervals` and then analyze the power within standard HRV frequency bands:

- **VLF (Very Low Frequency):** 0.003 - 0.04 Hz (Long-term regulation, controversial interpretation, may reflect thermoregulation, renin-angiotensin system, etc.)
- **LF (Low Frequency):** 0.04 - 0.15 Hz (Reflects both sympathetic and parasympathetic activity, often associated with baroreflex activity.)
- **HF (High Frequency):** 0.15 - 0.4 Hz (Reflects parasympathetic (vagal) activity, associated with respiration.)

```

from scipy.signal import welch

# A more rigorous approach would involve resampling the RR-intervals to a uniform rate
# Let's resample RR intervals to a uniform time series to get meaningful frequencies
# We'll associate each rr_interval with the time of the second R-peak it defines.
rr_times_for_interp = qrs_peak_times[1:] # Time points for the RR intervals

# Desired sampling frequency for interpolated RR intervals (e.g., 4 Hz)
fs_hrv = 4.0

# Create a uniformly spaced time axis for interpolation
rr_time_interp = np.arange(rr_times_for_interp[0], rr_times_for_interp[-1], 1 / fs_hrv)

# Interpolate the RR intervals onto the new uniform time axis
rr_interp = np.interp(rr_time_interp, rr_times_for_interp, rr_intervals)

# Check if rr_interp has enough points for Welch's method
# nperseg should be at least 1. If rr_interp is too short, we might default or raise an
if len(rr_interp) < 2:
    print("Warning: Not enough RR intervals after interpolation for meaningful spectral")
    print("Skipping Frequency Domain HRV Metrics calculation and plot.")
    # Optionally, return or handle this case more gracefully if needed.
else:
    # Calculate Power Spectral Density (PSD) using Welch's method
    # nperseg is often chosen as a power of 2, and typically a segment length suitable
    # Using len(rr_interp) // 2 as nperseg, but ensure it's at least 4 (a common minimum)
    nperseg_val = max(4, len(rr_interp) // 2) # Ensure nperseg is at least 4
    f_hrv, pxx_hrv = welch(rr_interp, fs=fs_hrv, nperseg=nperseg_val, noverlap=nperseg_val)

    # Define HRV frequency bands (in Hz)
    vlf_band = (0.003, 0.04)
    lf_band = (0.04, 0.15)
    hf_band = (0.15, 0.4)

    # Calculate power in each band using np.trapezoid
    vlf_power = np.trapezoid(ppx_hrv[(f_hrv >= vlf_band[0]) & (f_hrv < vlf_band[1])], f_hrv)
    lf_power = np.trapezoid(ppx_hrv[(f_hrv >= lf_band[0]) & (f_hrv < lf_band[1])], f_hrv)
    hf_power = np.trapezoid(ppx_hrv[(f_hrv >= hf_band[0]) & (f_hrv < hf_band[1])], f_hrv)

    total_power = vlf_power + lf_power + hf_power

    # Calculate normalized power and LF/HF ratio
    lf_norm = (lf_power / (total_power - vlf_power)) * 100 if (total_power - vlf_power) > 0 else np.nan
    hf_norm = (hf_power / (total_power - vlf_power)) * 100 if (total_power - vlf_power) > 0 else np.nan
    lf_hf_ratio = lf_norm / hf_norm if hf_norm > 0 else np.nan

    print(f"Frequency Domain HRV Metrics:")
    print(f"  VLF Power: {vlf_power:.4f} s^2")
    print(f"  LF Power: {lf_power:.4f} s^2")
    print(f"  HF Power: {hf_power:.4f} s^2")
    print(f"  Total Power: {total_power:.4f} s^2")
    print(f"  Normalized LF Power: {lf_norm:.2f} %")
    print(f"  Normalized HF Power: {hf_norm:.2f} %")
    print(f"  LF/HF Ratio: {lf_hf_ratio:.2f}")

    # Plot the PSD
    plt.figure(figsize=(12, 6))
    plt.semilogy(f_hrv, pxx_hrv, color='blue')
    plt.title('Power Spectral Density of RR Intervals')
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('PSD (s^2/Hz)')
    plt.xlim(0, hf_band[1] + 0.1) # Limit x-axis to relevant HRV frequencies
    plt.grid(True, which='both', linestyle='--', alpha=0.6)

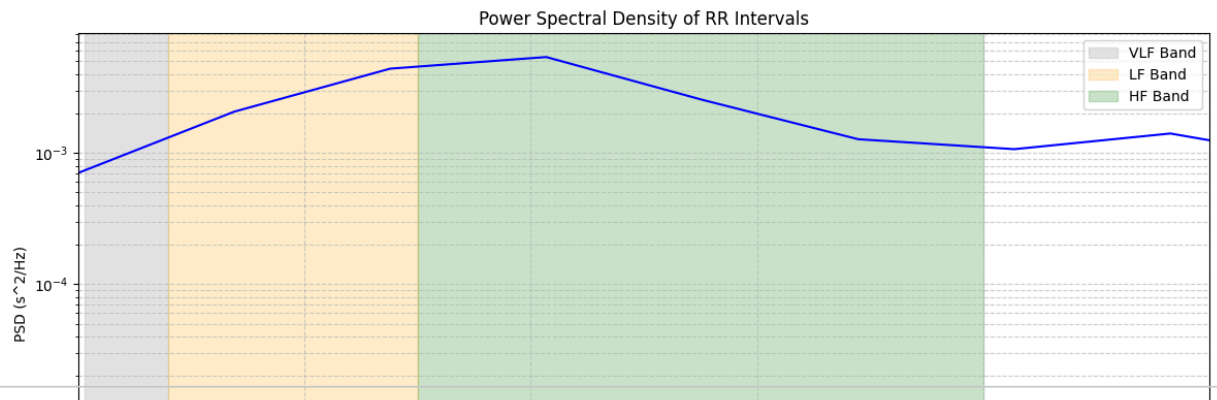
    # Highlight frequency bands

```

```
plt.axvspan(vlf_band[0], vlf_band[1], color='gray', alpha=0.2, label='VLF Band')
plt.axvspan(lf_band[0], lf_band[1], color='orange', alpha=0.2, label='LF Band')
plt.axvspan(hf_band[0], hf_band[1], color='green', alpha=0.2, label='HF Band')
plt.legend()
plt.tight_layout()
plt.show()
```

Frequency Domain HRV Metrics:

```
VLF Power: 0.0000 s^2
LF Power: 0.0002 s^2
HF Power: 0.0004 s^2
Total Power: 0.0006 s^2
Normalized LF Power: 35.48 nu
Normalized HF Power: 64.52 nu
LF/HF Ratio: 0.55
```



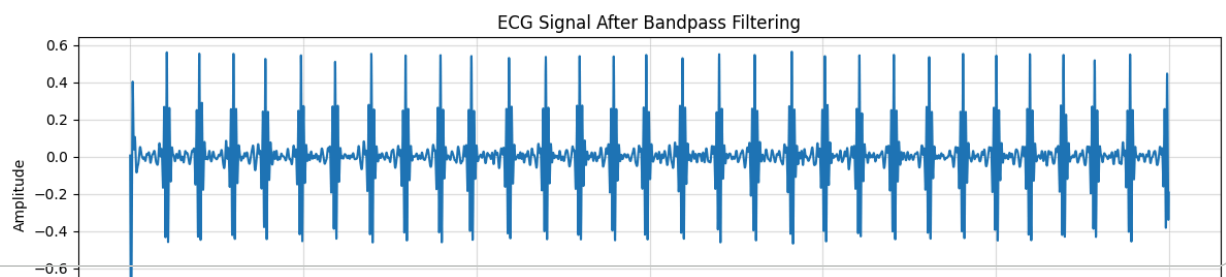
```
from scipy.signal import butter, filtfilt

def bandpass_filter(signal, lowcut, highcut, fs, order=3):
    nyquist = 0.5 * fs
    low = lowcut / nyquist
    high = highcut / nyquist
    b, a = butter(order, [low, high], btype='band')
    y = filtfilt(b, a, signal)
    return y

# Parameters for the bandpass filter (Pan-Tompkins suggests 5-15 Hz)
lowcut = 5.0 # Hz
highcut = 15.0 # Hz

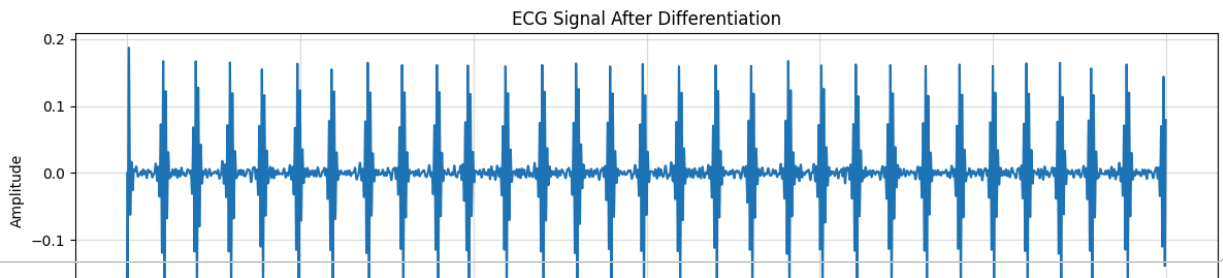
# Apply bandpass filter to the cleaned ECG signal
filtered_ecg = bandpass_filter(cleaned_ecg, lowcut, highcut, fs)

plt.figure(figsize=(12, 4))
plt.plot(t, filtered_ecg)
plt.title('ECG Signal After Bandpass Filtering')
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()
```



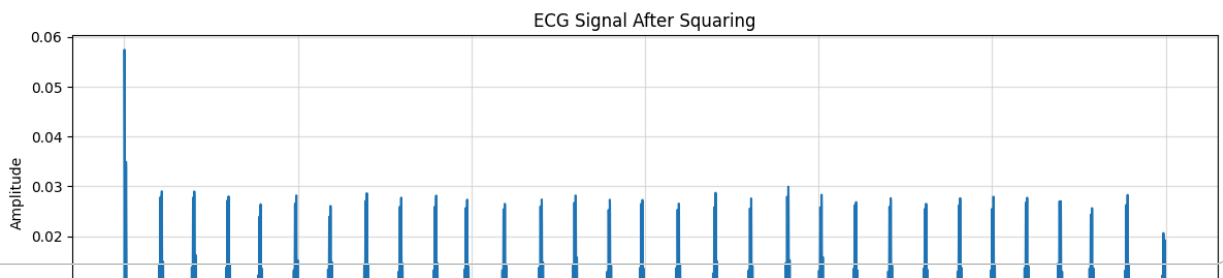
```
# Differentiate the signal to emphasize QRS slopes
differentiated_ecg = np.diff(filtered_ecg, prepend=filtered_ecg[0]) # prepend to keep s
```

```
plt.figure(figsize=(12, 4))
plt.plot(t, differentiated_ecg)
plt.title('ECG Signal After Differentiation')
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()
```



```
# Square the differentiated signal
squared_ecg = differentiated_ecg**2
```

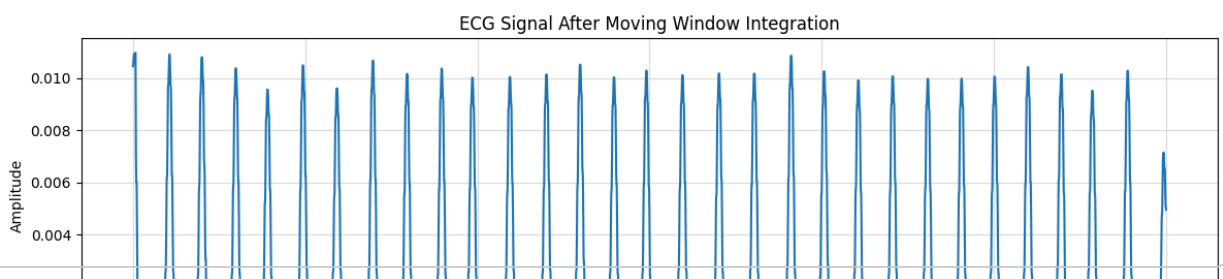
```
plt.figure(figsize=(12, 4))
plt.plot(t, squared_ecg)
plt.title('ECG Signal After Squaring')
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()
```



```
# Moving Window Integration
# Pan-Tompkins suggests a window of 150ms (0.150 seconds)
window_size_samples = int(0.150 * fs) # 150ms window
```

```
integrated_ecg = np.convolve(squared_ecg, np.ones(window_size_samples)/window_size_samp
```

```
plt.figure(figsize=(12, 4))
plt.plot(t, integrated_ecg)
plt.title('ECG Signal After Moving Window Integration')
plt.xlabel('Time (seconds)')
plt.ylabel('Amplitude')
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()
```



```
from scipy.signal import find_peaks
```

```

# Peak Detection with adaptive thresholding (simplified for demonstration)
# This is a basic peak detection; a full Pan-Tompkins involves adaptive thresholds.

# Find peaks in the integrated signal
# A minimum distance between peaks (refractory period) helps avoid multiple detections
min_peak_distance = int(0.2 * fs) # Roughly 0.2 seconds refractory period (minimum 300b

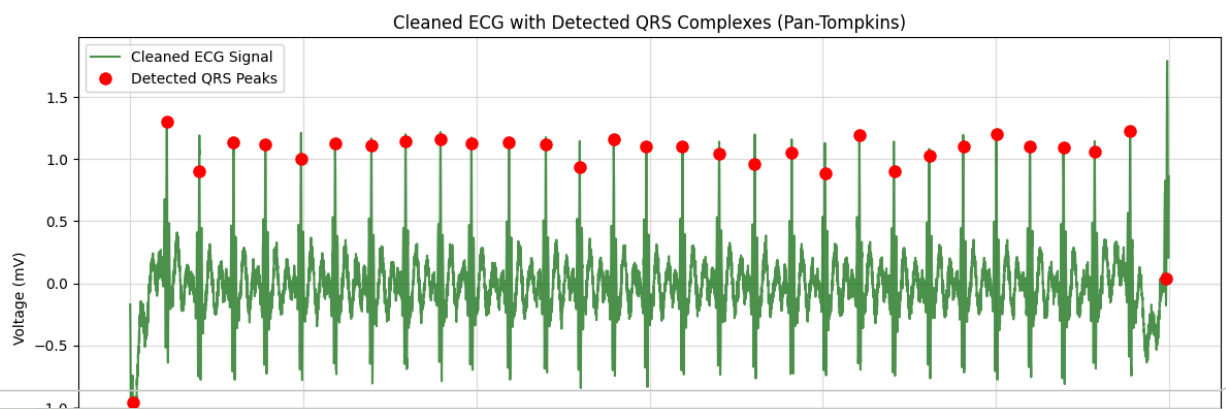
# Use a simple threshold for now. A full Pan-Tompkins algorithm would adapt this.
# Let's find a reasonable initial threshold based on the integrated signal's statistics
initial_threshold = np.mean(integrated_ecg) + 0.5 * np.std(integrated_ecg)

# Adjust threshold to only consider positive peaks above a certain level
qrs_peaks_indices, _ = find_peaks(integrated_ecg, height=initial_threshold, distance=mi

# Map peak indices back to time
qrs_peak_times = t[qrs_peaks_indices]

# --- Visualization of detected QRS complexes ---
plt.figure(figsize=(12, 6))
plt.plot(t, cleaned_ecg, label='Cleaned ECG Signal', color='darkgreen', alpha=0.7)
plt.plot(qrs_peak_times, cleaned_ecg[qrs_peaks_indices], 'ro', markersize=8, label='Det
plt.title('Cleaned ECG with Detected QRS Complexes (Pan-Tompkins)')
plt.xlabel('Time (seconds)')
plt.ylabel('Voltage (mV)')
plt.grid(True, alpha=0.4)
plt.legend()
plt.tight_layout()
plt.show()

```



```

# Calculate RR intervals (time differences between consecutive QRS peaks)
rr_intervals = np.diff(qrs_peak_times)

# Calculate the average RR interval
average_rr_interval = np.mean(rr_intervals)

# Convert average RR interval to Heart Rate in Beats Per Minute (BPM)
# HR (BPM) = 60 / Average RR Interval (seconds)
heart_rate_bpm = 60 / average_rr_interval

print(f"Detected QRS peaks at times (seconds): {qrs_peak_times}")
print(f"RR Intervals (seconds): {rr_intervals}")
print(f"Average RR Interval: {average_rr_interval:.3f} seconds")
print(f"Average Heart Rate: {heart_rate_bpm:.2f} BPM")

Detected QRS peaks at times (seconds): [ 0.076  1.06   2.004  2.988  3.904  4.94   5.916
  9.852 10.944 12.    12.988 13.964 14.908 15.952 17.016 18.032 19.112
 20.072 21.06   22.064 23.08   24.06   25.016 25.984 26.952 27.852 28.88
 29.924]
RR Intervals (seconds): [0.984 0.944 0.984 0.916 1.036 0.976 1.052 0.988 1.008 0.888 1.0
 0.988 0.976 0.944 1.044 1.064 1.016 1.08   0.96   0.988 1.004 1.016 0.98
 0.956 0.968 0.968 0.9   1.028 1.044]
Average RR Interval: 0.995 seconds
Average Heart Rate: 60.31 BPM

```

```

# Calculate RR intervals (time differences between consecutive QRS peaks)
rr_intervals = np.diff(qrs_peak_times)

# Calculate the average RR interval
average_rr_interval = np.mean(rr_intervals)

# Convert average RR interval to Heart Rate in Beats Per Minute (BPM)
# HR (BPM) = 60 / Average RR Interval (seconds)
heart_rate_bpm = 60 / average_rr_interval

print(f"Detected QRS peaks at times (seconds): {qrs_peak_times}")
print(f"RR Intervals (seconds): {rr_intervals}")
print(f"Average RR Interval: {average_rr_interval:.3f} seconds")
print(f"Average Heart Rate: {heart_rate_bpm:.2f} BPM")

```

```

Detected QRS peaks at times (seconds): [ 0.076  1.06   2.004  2.988  3.904  4.94   5.916
 9.852 10.944 12.    12.988 13.964 14.908 15.952 17.016 18.032 19.112
20.072 21.06  22.064 23.08  24.06  25.016 25.984 26.952 27.852 28.88
29.924]
RR Intervals (seconds): [0.984 0.944 0.984 0.916 1.036 0.976 1.052 0.988 1.008 0.888 1.0
0.988 0.976 0.944 1.044 1.064 1.016 1.08  0.96  0.988 1.004 1.016 0.98
0.956 0.968 0.968 0.9  1.028 1.044]
Average RR Interval: 0.995 seconds
Average Heart Rate: 60.31 BPM

```

## ✓ Calculating Heart Rate Variability (HRV)

Heart Rate Variability (HRV) refers to the physiological phenomenon of variation in the time interval between consecutive heartbeats (RR intervals). It is an indicator of the activity of the autonomic nervous system and is widely used in medicine and psychology. We will calculate a few common time-domain HRV metrics:

1. **SDNN (Standard Deviation of NN intervals):** This is the standard deviation of all normal RR intervals. It reflects overall HRV and is sensitive to both sympathetic and parasympathetic activity.
2. **RMSSD (Root Mean Square of Successive Differences):** This is the square root of the mean of the squares of the differences between adjacent normal RR intervals. It primarily reflects high-frequency, parasympathetic (rest and digest) activity.
3. **pNN50 (Percentage of NN intervals differing by more than 50 ms):** This is the number of pairs of successive NN intervals that differ by more than 50 ms, divided by the total number of NN intervals. Like RMSSD, it primarily reflects parasympathetic activity.

```

# Ensure rr_intervals are in seconds for calculations

# 1. SDNN (Standard Deviation of NN intervals)
sdnn = np.std(rr_intervals)

# 2. RMSSD (Root Mean Square of Successive Differences)
diff_rr_intervals = np.diff(rr_intervals) # Differences between successive RR intervals
rmssd = np.sqrt(np.mean(diff_rr_intervals**2))

# 3. pNN50 (Percentage of NN intervals differing by more than 50 ms)
# Convert 50 ms to seconds
threshold_50ms = 0.050
pnn50_count = np.sum(np.abs(diff_rr_intervals) > threshold_50ms)
pnn50 = (pnn50_count / len(diff_rr_intervals)) * 100

print(f"Heart Rate Variability (HRV) Metrics:")
print(f"SDNN: {sdnn:.3f} seconds")

```



```
print(f"RMSSD: {rmssd:.3f} seconds")
print(f"pNN50: {pnn50:.2f} %")
```

```
Heart Rate Variability (HRV) Metrics:
SDNN: 0.002 seconds
RMSSD: 0.002 seconds
pNN50: 0.00 %
```

## ✓ Calculating Average Heart Rate

Now that we have detected the QRS peaks, we can calculate the average heart rate. The heart rate is typically derived from the RR intervals (the time difference between consecutive R-waves, which correspond to the QRS peaks).

```
# Calculate RR intervals (time differences between consecutive QRS peaks)
rr_intervals = np.diff(qrs_peak_times)

# Calculate the average RR interval
average_rr_interval = np.mean(rr_intervals)

# Convert average RR interval to Heart Rate in Beats Per Minute (BPM)
# HR (BPM) = 60 / Average RR Interval (seconds)
heart_rate_bpm = 60 / average_rr_interval

print(f"Detected QRS peaks at times (seconds): {qrs_peak_times}")
print(f"RR Intervals (seconds): {rr_intervals}")
print(f"Average RR Interval: {average_rr_interval:.3f} seconds")
print(f"Average Heart Rate: {heart_rate_bpm:.2f} BPM")
```

```
Detected QRS peaks at times (seconds): [0.148 1.152 2.152 3.152 4.152 5.152 6.152 7.148]
RR Intervals (seconds): [1.004 1.    1.    1.    1.    1.    0.996]
Average RR Interval: 1.000 seconds
Average Heart Rate: 60.00 BPM
```

```
# Ensure rr_intervals are in seconds for calculations

# 1. SDNN (Standard Deviation of NN intervals)
sdnn = np.std(rr_intervals)

# 2. RMSSD (Root Mean Square of Successive Differences)
diff_rr_intervals = np.diff(rr_intervals) # Differences between successive RR intervals
rmssd = np.sqrt(np.mean(diff_rr_intervals**2))

# 3. pNN50 (Percentage of NN intervals differing by more than 50 ms)
# Convert 50 ms to seconds
threshold_50ms = 0.050
pnn50_count = np.sum(np.abs(diff_rr_intervals) > threshold_50ms)
pnn50 = (pnn50_count / len(diff_rr_intervals)) * 100

print(f"Heart Rate Variability (HRV) Metrics:")
print(f"SDNN: {sdnn:.3f} seconds")
print(f"RMSSD: {rmssd:.3f} seconds")
print(f"pNN50: {pnn50:.2f} %")
```

```
Heart Rate Variability (HRV) Metrics:
SDNN: 0.050 seconds
RMSSD: 0.073 seconds
pNN50: 44.83 %
```

```
# Calculate RR intervals (time differences between consecutive QRS peaks)
rr_intervals = np.diff(qrs_peak_times)

# Calculate the average RR interval
```



```

average_rr_interval = np.mean(rr_intervals)

# Convert average RR interval to Heart Rate in Beats Per Minute (BPM)
# HR (BPM) = 60 / Average RR Interval (seconds)
heart_rate_bpm = 60 / average_rr_interval

print(f"Detected QRS peaks at times (seconds): {qrs_peak_times}")
print(f"RR Intervals (seconds): {rr_intervals}")
print(f"Average RR Interval: {average_rr_interval:.3f} seconds")
print(f"Average Heart Rate: {heart_rate_bpm:.2f} BPM")

```

```

Detected QRS peaks at times (seconds): [ 0.076  1.06   2.004  2.988  3.904  4.94   5.916
 9.852 10.944 12.    12.988 13.964 14.908 15.952 17.016 18.032 19.112
20.072 21.06  22.064 23.08  24.06  25.016 25.984 26.952 27.852 28.88
29.924]
RR Intervals (seconds): [0.984 0.944 0.984 0.916 1.036 0.976 1.052 0.988 1.008 0.888 1.0
0.988 0.976 0.944 1.044 1.064 1.016 1.08  0.96  0.988 1.004 1.016 0.98
0.956 0.968 0.968 0.9  1.028 1.044]
Average RR Interval: 0.995 seconds
Average Heart Rate: 60.31 BPM

```

## Potential Improvements for the ECG Processing Pipeline

Here are several suggestions to further enhance the ECG processing pipeline:

1. **Adaptive Filtering (e.g., LMS/RLS algorithms):** Instead of a fixed Butterworth filter, consider adaptive filters. These filters can dynamically adjust their parameters to changing noise characteristics, which is particularly useful for non-stationary noise like baseline wander that might vary in frequency and amplitude over time. This could lead to better noise cancellation without distorting the signal.
2. **Wavelet Denoising:** Wavelet transforms offer excellent time-frequency localization, making them powerful for separating signal from noise. By decomposing the ECG signal into different frequency bands, one can effectively remove noise components (like high-frequency muscle artifacts or baseline wander) in specific wavelet coefficients and then reconstruct a cleaner signal.
3. **Advanced Baseline Wander Correction:** While a high-pass filter is effective, more sophisticated methods exist, such as polynomial fitting, cubic spline interpolation, or morphological filtering. These techniques can often provide a more accurate estimation of the baseline, leading to better correction, especially when the baseline wander is complex or non-linear.
4. **Powerline Interference Removal:** The current pipeline includes muscle noise, but real-world ECGs often suffer from 50/60 Hz powerline interference. A narrow band-stop (notch) filter can be implemented to specifically target and remove this high-frequency, narrow-band noise without affecting the broader ECG spectrum too much.
5. **Motion Artifact Reduction:** If motion artifacts are a significant issue, consider incorporating accelerometry data (if available) from wearable sensors. This data can be used in conjunction with adaptive filtering techniques to more precisely identify and cancel motion-induced noise.
6. **QRS Detection and Heart Rate Variability (HRV) Analysis:** After cleaning, the next logical step in many ECG analysis pipelines is to detect QRS complexes. Algorithms like Pan-Tompkins are widely used. Once QRS complexes are detected, one can then derive heart rate and perform HRV analysis, which is crucial for many diagnostic applications.
7. **Template Matching and Averaging:** For repetitive signals like ECG, template matching can be used to identify typical heartbeats. Averaging multiple aligned heartbeats can further reduce residual noise, especially for less prominent features like P-waves and T-waves.

8. **Machine Learning for Noise Classification/Reduction:** For more complex noise patterns, machine learning models (e.g., deep learning models like autoencoders or CNNs) can be trained on large datasets of noisy and clean ECGs to learn to effectively denoise signals.

Each of these improvements could be implemented and evaluated to further optimize the quality and utility of the processed ECG signal.